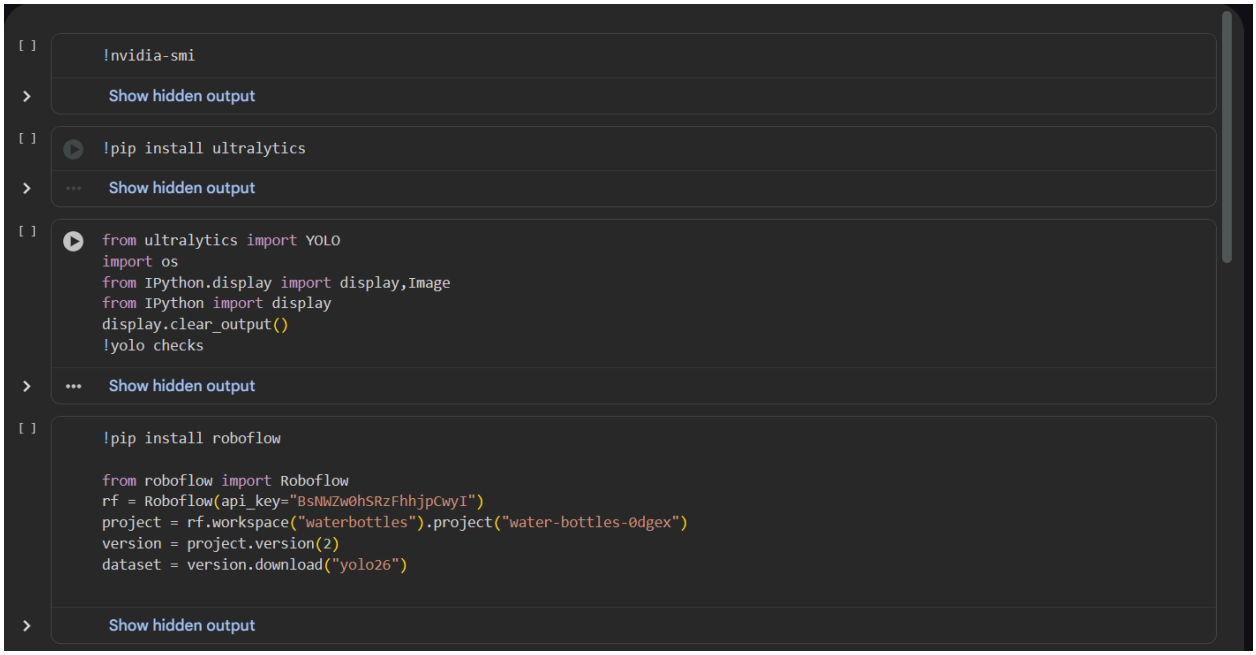
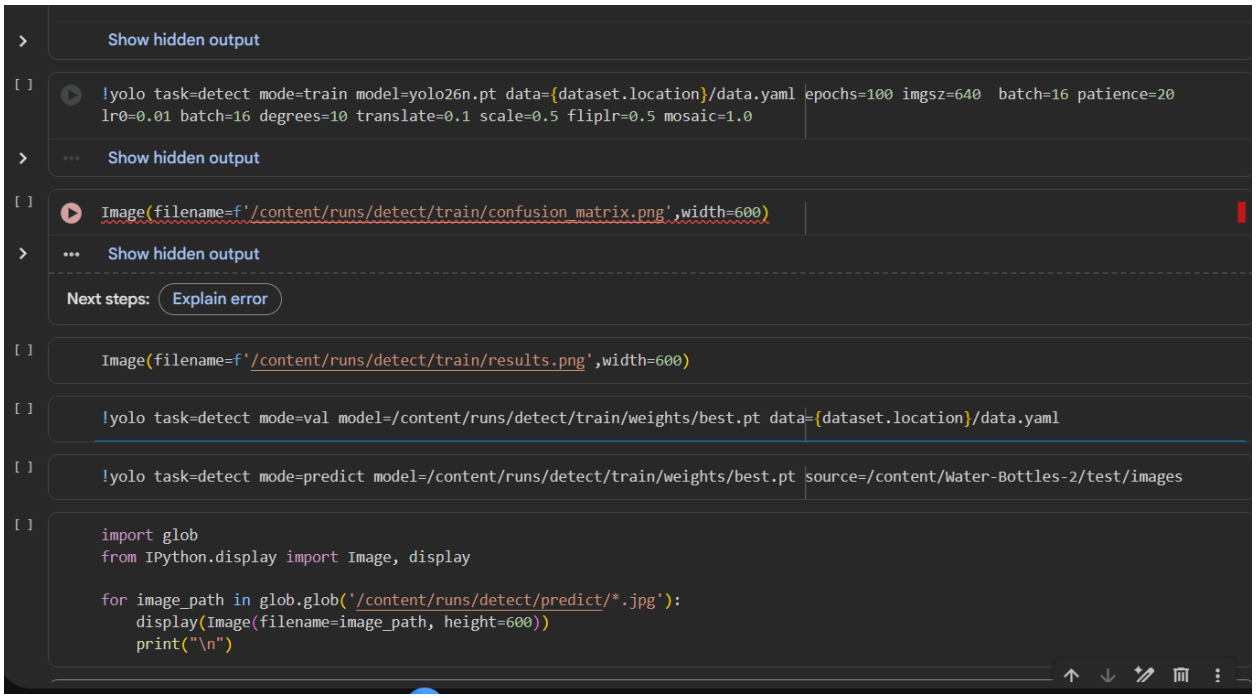
Documentation for the second day by Youssef Eslam from Softwear about object detection

Overview: The object detection part of the project focuses on training a YOLO26n (Nano) model to detect water bottles in images and live camera footage. A dataset containing water bottle images is used to train YOLO26n in Google Colab; manual hyperparameter tuning is used to improve the model's performance. Parameters such as learning rate, batch size, image size, epochs, and data augmentation are adjusted and compared; then the model is validated and tested to measure its accuracy and check its predictions. Finally, the trained model is used with a live camera feed, where it detects bottles in real time and displays bounding boxes around them using my phone as a webcam on its Wi-Fi.

First Google Colab code:



```
[ ]  !nvidia-smi
     Show hidden output

[ ]  !pip install ultralytics
     Show hidden output

[ ]  from ultralytics import YOLO
     import os
     from IPython.display import display,Image
     from IPython import display
     display.clear_output()
     !yolo checks
     Show hidden output

[ ]  !pip install roboflow

     from roboflow import Roboflow
     rf = Roboflow(api_key="BsNWZw0hSRzFhhjpCwyI")
     project = rf.workspace("waterbottles").project("water-bottles-0dgex")
     version = project.version(2)
     dataset = version.download("yolo26")

     Show hidden output
```



```
     Show hidden output

[ ]  !yolo task=detect mode=train model=yolo26n.pt data={dataset.location}/data.yaml epochs=100 imgsz=640  batch=16 patience=20
     lr0=0.01 batch=16 degrees=10 translate=0.1 scale=0.5 fliplr=0.5 mosaic=1.0
     Show hidden output

[ ]  Image(filename=f'/content/runs/detect/train/confusion_matrix.png',width=600)
     Show hidden output

     Next steps: Explain error

[ ]  Image(filename=f'/content/runs/detect/train/results.png',width=600)

[ ]  !yolo task=detect mode=val model=/content/runs/detect/train/weights/best.pt data={dataset.location}/data.yaml

[ ]  !yolo task=detect mode=predict model=/content/runs/detect/train/weights/best.pt source=/content/Water-Bottles-2/test/images

[ ]  import glob
     from IPython.display import Image, display

     for image_path in glob.glob('/content/runs/detect/predict/*.jpg'):
         display(Image(filename=image_path, height=600))
         print("\n")
```

Code explanation for future reference :

!pip install ultralytics:

Installs the ultralytics library, witch contain yolo model.

```
from ultralytics import YOLO
import os
from IPython.display import display, Image
from IPython import display
display.clear_output()
!yolo checks:
```

Import yolo functions and gives it the ability to use images and checks if it works

```
!pip install roboflow
from roboflow import Roboflow
rf = Roboflow(api_key="BsNWZw0hSRzFhhjpCwyI")
project = rf.workspace("waterbottles").project("water-bottles-0dgex")
version = projectversion(2)
dataset = versiondownload("yolo26"):
```

Install the roboflow library, install the waterbottles project and datasheet from roboflow, and install version 2 and yolo26

```
!yolo task=detect mode=train model=yolo26n.pt data={dataset.location}/data.yaml
epochs=100 imgsz=640  batch=16 patience=20
lr0=0.01 batch=16 degrees=10 translate=0.1 scale=0.5 fliplr=0.5 mosaic=1.0:
```

Here is the training part of YOLO26 and the hyperparameters used:

Model: nano, as it's fast, and I want fast image detection in real time so the car can detect it and move towards it even if the bottle is moving
Epochs: it is how many passes in training; here it is 100. It's the maximum; it won't run all, as I put patience=20
Imgsz: the image size used in training; I used a large one for more details
Batch: how many images are used at once before updating the model; here it is 16
patience: if the model didn't improve for 20 epochs, it will stop the training so as not to waste time and resources
Lr0:its learning rate; it's how much the model changes itself in the internal weights

rest: affects the image that the model will see from degree shift and translation left and right and size and flipping the image and putting multiple images; this allows for different scenarios for the model to see

```
Image(filename=f'/content/runs/detect/train/confusion_matrix.png',width=600)
Image(filename=f'/content/runs/detect/train/results.png',width=600):
```

Show graphs for debugging

```
!yolo task=detect mode=val model=/content/runs/detect/train/weights/best.pt
data={dataset.location}/data.yaml:
```

After training, this is for validation using validation data with the best model.

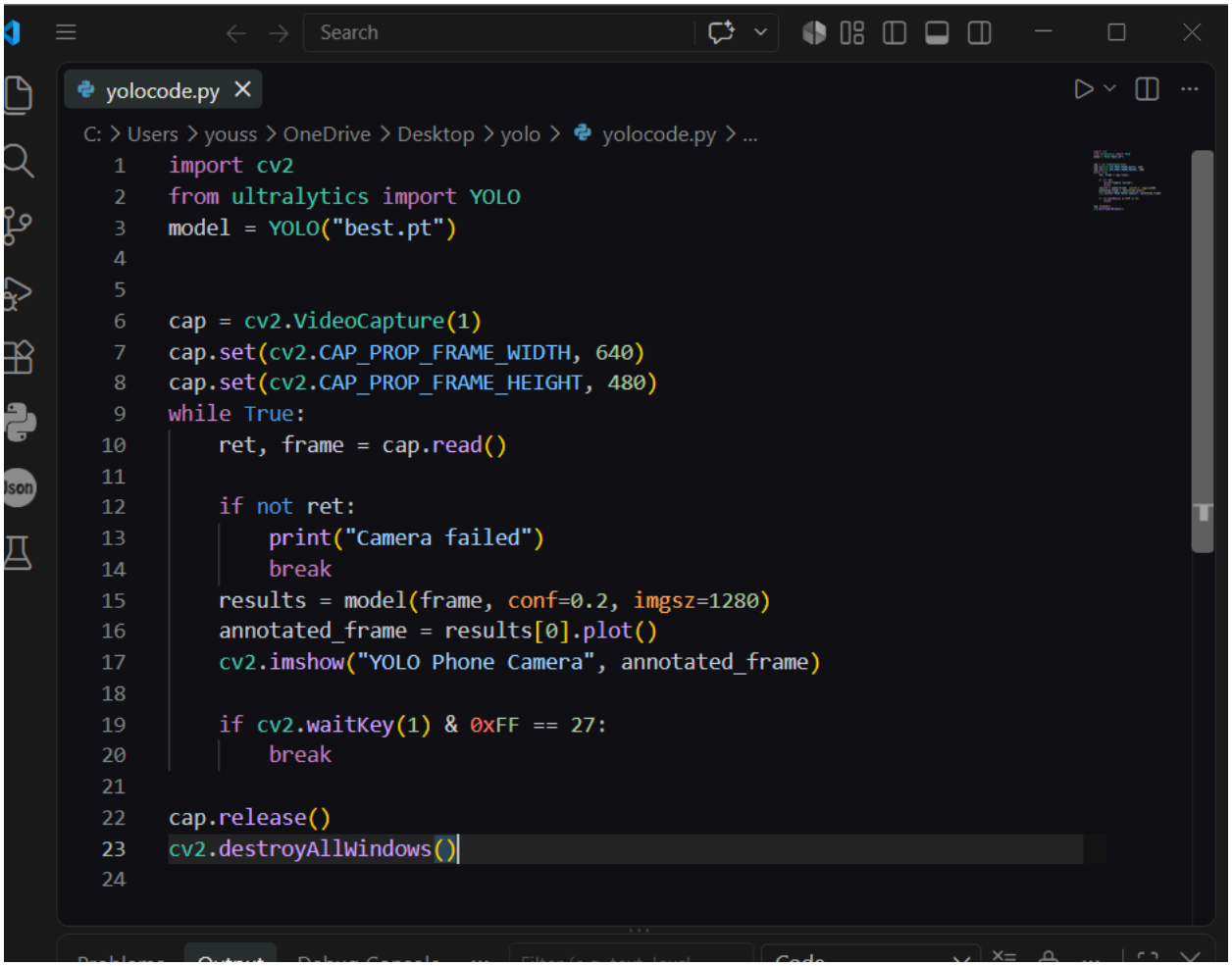
```
!yolo task=detect mode=predict model=/content/runs/detect/train/weights/best.pt
source=/content/Water-Bottles-2/test/images:
```

This uses the trained model to predict from prediction data.

```
import glob
from IPython.display import Image, display
for image_path in glob.glob('/content/runs/detect/predict/*.jpg'):
    display(Image(filename=image_path, height=600))
    print("\n"):
```

Shows predictions of bottles on screen

Second VS Code code:



Code explanation for future reference:

```
import cv2
from ultralytics import YOLO
model = YOLO("best.pt"):
```

Import libraries OpenCV for the camera and yolo for the model
It reads my model name, best.pt

```
cap = cv2.VideoCapture(2):
```

This opens camera number 2
Notes: Camera number 2 is my phone, which I used as a webcam to mount it on the car using an app called Camo Camera.

```
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480):
```

This is for camera resolution

```
while True:
    ret, frame = cap.read()

    if not ret:
        print("Camera failed")
        Break:
```

This loops through the camera for each frame, and if it didn't send camera failed

```
    results = model(frame, conf=0.2, imgsz=1280):
```

This is the trained model; it shows only when it is sure over 0.2, and the resolution is 1280
I chose this high resolution as the bottle will be far away and i want it to detect it

```
    annotated_frame = results[0].plot():
```

It is used to draw the box

```
    cv2.imshow("YOLO Phone Camera", annotated_frame):
```

Open camera window

```
    if cv2.waitKey(1) & 0xFF == 27:
        Break:
```

Checks when I press the Esc key

cap.release()
cv2.destroyAllWindows():

Closes window at end of code

Points to clarify :

How I would annotate the dataset from scratch and suggest a website to do this job:

I would use Roboflow Annotate; it provides tools for annotation, and I will create a project, open each image, draw a box around my object, give it a class called water_bottle, and in the end I will split the dataset into training, validation, and test and download it.

Which YOLO model’s size you used and why:

I used YOLOv26n, the nano version, because I want fast real-time object detection, and its light and small model, and it's a simple task, as I detect one class.

How can you tune hyperparameters to improve model’s performance :

There is for ways: manual tuning, changing the hyperparameters by myself and checking results for different values until it's good; Grid Search, which chooses several values for each hyperparameter and tests every possible combination. Random Search: it chooses ranges of values and lets the algorithm randomly test different combinations. Bayesian Optimization: the algorithm learns from previous tests and uses that information to choose the next promising combination. I chose manual hyperparameter tuning because it requires fewer computational resources and it gives me direct control over the hyperparameters, allowing me to adjust them based on the model's performance and the requirements of my project. As discussed above, for each hyperparameter.

Videos:
https://drive.google.com/file/d/1GombVs48gHGfoEafPqwawv5jyWgnGT2o/view?usp=sharing